

 DAY 1 | 8:00 PM – 9:00 PM | 60 MIN

# API Serving & Containerized Inference

Transforming Trained Machine Learning Models into Production-Ready  
REST APIs with FastAPI & Docker

 FastAPI

 Docker Containers

 Pydantic Schema



# Why Serve ML Models via API?

## ⚠️ Direct Model File Risks

**Language Lock-in:** Model format (.pkl) requires Python code on client side.

**No Schema Enforcement:** Invalid input types crash the model in production.

**Version Chaos:** Hard to deploy updates without breaking client applications.

**No Scaling Isolation:** Model inference blocks UI/web application threads.

## ✅ REST API Solution

**Universal Interoperability:** Any app (iOS, React, Java) speaks HTTP/JSON.

**Strict Input Validation:** Pydantic rejects bad payloads before inference.

**Route Versioning:** Clean `/v1/predict` and `/v2/predict` migration paths.

**Independent Scaling:** Scale API inference instances separately from web servers.



# The Inference Request Lifecycle



## 1. Client Request

Web or mobile client sends a standard HTTP POST JSON payload to the endpoint.

```
{
  "age": 35,
  "income": 75000,
  "credit_score": 720
}
```



## 2. FastAPI & Pydantic

Validates data types, enforces range checks (e.g.  $\text{age} \geq 18$ ), and parses into Python dict.

✓ Rejects malformed requests with 422 Error



## 3. Model & Response

Loads model features, runs `predict_proba()`, and returns a validated JSON response.

```
{
  "prediction": "No Default",
  "probability": 0.10
}
```



# FastAPI & Pydantic Schemas

```
from pydantic import BaseModel, Field from fastapi import FastAPI class
PredictRequest(BaseModel):    age: int = Field(..., ge=18, le=120)
    income: float = Field(..., gt=0)    credit_score: int = Field(...,
ge=300, le=850) class PredictResponse(BaseModel):    prediction: str
    probability: float app = FastAPI(title="Bank Risk ML API")
```

## Automatic Data Validation

Pydantic enforces strict type checking and range limits automatically. Bad data is rejected before hitting your ML code.

## Interactive OpenAPI Docs

FastAPI automatically generates interactive Swagger UI documentation at <http://localhost:8000/docs> for easy testing.

## High Performance Async

Built on Starlette and ASGI standards, delivering performance on par with NodeJS and Go.



# Two Golden Rules of ML APIs

## GOLDEN RULE #1

### 🕒 Load Model ONCE at Startup

Use FastAPI's `@asynccontextmanager` lifespan handler to load the model into memory during server boot.

```
# NEVER load model inside predict route! @asynccontextmanager async
def lifespan(app: FastAPI):    global model    model =
    joblib.load("model.pkl")    yield
```

**Why?** Reloading a 500MB model per request adds 200ms+ latency per call!

## GOLDEN RULE #2

### 💓 Always Expose `/health`

Provide a lightweight endpoint that returns model readiness status for orchestrators.

```
@app.get("/health") async def health_check():    return {
    "status": "healthy",        "model_loaded": model is not
    None                        }
```

**Why?** Kubernetes & AWS Load Balancers rely on this to route traffic safely.



# Shipping Code with Docker

## Eliminating "Works on My Machine"

Docker packages your Python application, ML model artifacts, C++ system libraries, and OS configurations into a single, lightweight, and reproducible container image.

### Image (Blueprint / Recipe)

An immutable, read-only package containing your application code, Python dependencies, model weights (.pkl / .onnx), and OS runtime.

- Read-Only
- Versioned (v1.0.0)
- Registry Shareable

### Container (Running Instance)

An isolated, runnable process instance instantiated from an image. You can launch 100 identical containers instantly across your cluster.

- Process Isolated
- Port-Mapped
- Instant Auto-Scale

#### PRO TIP Stateless Container Pattern

Never store state on local container disk. Save predictions and logs to databases or cloud storage (S3).





# The Docker Layer Cache Trick

-  **FROM python:3.11-slim — Base OS Layer:** Downloads minimal Linux & Python runtime (Changes standard base environment).
-  **COPY requirements.txt . — Dependency Blueprint:** Copies library manifests before any source code (Changes rarely).
-  **RUN pip install --no-cache-dir -r requirements.txt:** Installs heavy ML libraries like pandas & scikit-learn. **CACHED KEY STEP**
-  **COPY app/ ./app/ — Application Source Code:** Copies Python API code. Placed near the bottom because app code changes frequently!
-  **CMD ["uvicorn", "app.main:app"]:** Sets startup command when container boots up.

 **Pro-Tip:** If code changes, Docker re-uses cached layers up to the COPY line, skipping 5+ minutes of pip installs!





# Production Dockerfile Blueprint

| Instruction | Example Command                       | Production Purpose & Best Practice                                             |
|-------------|---------------------------------------|--------------------------------------------------------------------------------|
| FROM        | <code>python:3.11-slim</code>         | Minimizes image size (~150MB vs ~1GB full image) to reduce attack surface.     |
| WORKDIR     | <code>WORKDIR /app</code>             | Sets clean working directory inside container for all subsequent commands.     |
| COPY / RUN  | <code>COPY requirements.txt .</code>  | Leverages Docker cache by installing requirements prior to application code.   |
| USER        | <code>USER appuser</code>             | Runs app under non-root user account for enterprise security compliance.       |
| EXPOSE      | <code>EXPOSE 8000</code>              | Documents default container networking port for documentation & orchestration. |
| HEALTHCHECK | <code>HEALTHCHECK CMD curl ...</code> | Allows container engine to auto-restart hung or unhealthy containers.          |



# Essential Docker Commands



## 1. Build Image

Compiles Dockerfile into a local image tagged with name and version.

```
docker build \  
-t ml-api:1.0.0 .
```



## 2. Run Container

Launches container in detached mode, mapping host port 8000.

```
docker run -d \  
-p 8000:8000 \  
--name ml-api ml-api:1.0.0
```



## 3. Verify & Monitor

Inspects live logs and sends test prediction requests via curl.

```
docker logs ml-api  
curl http://localhost:8000/health
```



# Inference Serving Patterns

| Serving Pattern       | Primary Target Use Case                             | Latency            | Throughput |
|-----------------------|-----------------------------------------------------|--------------------|------------|
| Synchronous REST API  | Real-time single predictions (Web/Mobile Apps)      | Low (< 100ms)      | Medium     |
| Asynchronous Queue    | High-volume non-urgent tasks (Document processing)  | Medium (Seconds)   | High       |
| Batch Offline Scoring | Bulk nightly predictions (Credit scoring databases) | High (Minutes)     | Very High  |
| Streaming Inference   | Real-time event streams (Fraud detection / Kafka)   | Ultra-Low (< 10ms) | Maximum    |

 **Rule of Thumb:** Choose Synchronous REST (FastAPI) for interactive applications, and Batch/Streaming for background pipelines.



# Hands-On Lab Workflow

1

## Inspect Code

Review `main.py` and `schemas.py` in the `app/` directory.

2

## Run Local API

Execute `uvicorn app.main:app`  
`--reload` & test `/docs` UI.

3

## Build Docker

Run `docker build -t my-ml-api .` in terminal window.

4

## Deploy & Test

Launch container with `docker run -p 8000:8000` and ping `/health`.



# Key Takeaways & Quiz

## ☰ Essential Takeaways

- **FastAPI:** Modern Python standard with auto async, docs, & Pydantic typing.
- **Model Startup:** Always load model once during app startup context.
- **Docker Caching:** Copy `requirements.txt` before application source code.
- **Health Endpoint:** Always expose `/health` for load balancer monitoring.

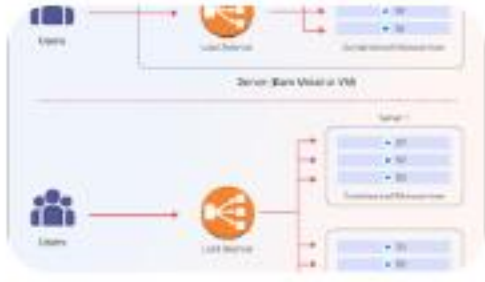
## 🔍 Quick Newbie Quiz

Why do we copy `requirements.txt` and run `pip install` before copying our application source code in a Dockerfile?

**Answer:** To maximize Docker layer caching! Because app code changes frequently while library dependencies rarely change, this avoids re-downloading heavy ML libraries on every code edit.



# Image Sources



<https://www.simform.com/wp-content/uploads/2023/12/Deploy-microservices-with-containers.png>

Source: [www.simform.com](https://www.simform.com)